Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192471039
Student Name	K R Gayana

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])        // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Code of Conduct

I **K R Gayana(192471039)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K R gayana Date: 13/10/2026

ANSWER

(A) IDENTIFY THE BASIC OPERATIONS AND TIME COMPLEXITIES OF M1–M5

ShopSphere is an e-commerce platform planning to expand its product catalogue to one million products. Each module must be analysed to determine whether it can handle the increased workload efficiently.

Module	Basic operation	Time complexity
M1 – Duplicate-Listing Detector	Compares product pairs to identify duplicates	$\Theta(n^2)$
M2 – API Rate Limiter	Performs operations over a sequence of doubling sizes	$\Theta(n)$
M3 – Product Search S1	Sequentially checks products	Best: $\Theta(1)$, Worst: $\Theta(n)$
M3 – Product Search S2	Searches a sorted product index using binary search	$\Theta(\log n)$
M4 – Recommendation Engine	Uses the recurrence $T(n)=4T(n/2)+n$	$\Theta(n^2)$
M5 – Sales Analytics Engine	Uses the recurrence $T(n)=2T(n/2)+n \log n$	$\Theta(n \log^2 n)$

Explanation:

• M1: The nested-loop approach compares each product with other products, producing a quadratic number of comparisons.

• M2: The total work forms a geometric sequence, giving linear growth.

• M3: Sequential search may examine every product, whereas binary search repeatedly halves a sorted search range.

• M4: Four subproblems of size n/2 are created, along with linear additional work.

• M5: Two subproblems of size n/2 are created, along with n log n additional work.

(B) DERIVE THE OPERATION COUNTS FOR M1 AND M2

M1 – Duplicate-Listing Detector

The algorithm compares every pair of products. The number of comparisons is

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

Using the sum of the first n−1 natural numbers,

$$C(n) = n(n-1)/2$$

Therefore, $C(n) = \Theta(n^2)$

Conclusion: The number of comparisons grows quadratically as the catalogue increases.

M2 – API Rate Limiter

According to the reference document, the operation count is

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For n = 1,000,000,

$$\lceil \log_2(1{,}000{,}000) \rceil = 20$$

Substituting, $A(n) = 2^{20} - 1$

$$A(n) = 1{,}048{,}576 - 1$$

$$A(n) = 1{,}048{,}575$$

Since $2^{\lceil \log_2 n \rceil}$ is bounded by a constant factor of n, the tight asymptotic complexity is

$$A(n) = \Theta(n)$$

Conclusion: M2 has linear time complexity and is suitable for further scalability, subject to the given processing assumptions.

(C) SOLVE THE RECURRENCE RELATION

Given:

$$T(n) = T(n/2) + 1, \quad T(1) = 1$$

Step 1: Expand the recurrence

$$T(n) = T(n/2) + 1$$

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/8) + 3$$

After k expansions,

$$T(n) = T(n/2^k) + k$$

Step 2: Find the stopping condition

The recurrence stops when the input size becomes 1.

$n/2^k = 1$

$n = 2^k$

Taking logarithm base 2, $k = \log_2 n$

Step 3: Substitute the value

$T(n) = T(1) + \log_2 n$

Since $T(1) = 1$, $T(n) = 1 + \log_2 n$

Therefore, the time complexity is $T(n) = \Theta(\log n)$

Conclusion: The recurrence has logarithmic time complexity because the input size is halved at every step.

(D) APPLY THE MASTER THEOREM TO M4 AND M5

The Master Theorem is used for recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

where a is the number of subproblems, n/b is each subproblem's size, and f(n) is the additional work.

1. M4 – Current Recommendation Engine

Given: $T(n) = 4T(n/2) + n$

Identify the values: $a = 4$, $b = 2$, $f(n) = n$

Calculate: $n^{(\log_b a)} = n^{(\log_2 4)} = n^2$

Since $f(n) = n$ grows more slowly than n^2, the first case of the Master Theorem applies.

$$T(n) = \Theta(n^2)$$

Result: The current recommendation engine has quadratic complexity and may become too slow for a large catalogue.

2. M4 – Proposed Recommendation Engine

The proposed design uses: T(n) = 2T(n/2) + n

Identify the values: a = 2, b = 2, f(n) = n

Calculate: n^(log_b a) = n^(log$_2$ 2) = n

Since f(n) = Θ(n), the second case of the Master Theorem applies.

T(n) = Θ(n log n)

Result: The proposed design improves the complexity from Θ(n^2) to Θ(n log n).

3. M5 – Sales Analytics Engine

Given: T(n) = 2T(n/2) + n log n

Identify the values: a = 2, b = 2, f(n) = n log n

Calculate: n^(log_b a) = n^(log$_2$ 2) = n

Here, f(n) = Θ(n log n). This is the second case of the Master Theorem with an additional logarithmic factor.

T(n) = Θ(n log^2 n)

Result: M5 has n log^2 n time complexity and is expected to meet the processing limit under the assumptions in the reference document.

(E) COMPARE THE BEST, AVERAGE, AND WORST CASES OF SEQUENTIAL SEARCH AND BINARY SEARCH

Sequential search

Sequential search checks elements one by one until the target is found or the list ends.

Case	Explanation	Time complexity
Best	Target is the first element	Θ(1)

Average	Target is typically found after examining a portion of the list	Θ(n)
Worst	Target is the last element or is absent	Θ(n)

Binary search

Binary search requires a sorted list. It compares the target with the middle element and repeatedly halves the search range.

Case	**Explanation**	**Time complexity**
Best	Target is the middle element in the first comparison	Θ(1)
Average	The search range is repeatedly halved	Θ(log n)
Worst	The target is found after the maximum number of halvings, or is absent	Θ(log n)

Comparison and conclusion: Binary search is more efficient than sequential search for large, sorted product catalogues. ShopSphere should retain the binary-search implementation (S2) and ensure that the product index remains sorted and updated.

(F) PROVE THAT $F(N)=3N^2+10N \log_2 N+50$ IS $\Theta(N^2)$, BUT NOT $O(N)$

GIVEN: $F(N) = 3N^2 + 10N \log_2 N + 50$

Step 1: Prove the lower bound

For $n \geq 1$, both $10n \log_2 n$ and 50 are non-negative.

Therefore, $f(n) \geq 3n^2$

Choose $c_1 = 3$. Then, $f(n) \geq c_1 n^2$

Step 2: Prove the upper bound

For $n \geq 1$, $\log_2 n \leq n$

Therefore, $10n \log_2 n \leq 10n^2$

Also, $50 \leq 50n^2$

Combining these inequalities,

$f(n) \leq 3n^2 + 10n^2 + 50n^2$

$f(n) \leq 63n^2$

Choose $c_2 = 63$. Thus, $f(n) \leq c_2 n^2$

Step 3: Apply the definition of Theta

We have established that, for every $n \geq 1$,

$$\mathbf{3n^2 \leq f(n) \leq 63n^2}$$

Taking $c_1 = 3$, $c_2 = 63$, and $n_0 = 1$, the formal definition of Theta is satisfied.

Hence, $f(n) = \Theta(n^2)$

Step 4: Prove that f(n) is not O(n)

Assume that $f(n) = O(n)$. Then there must be a constant $c > 0$ such that

$$f(n) \leq cn$$

for sufficiently large n.

However, since $f(n) \geq 3n^2$,

$3n^2 \leq cn$

Dividing both sides by $n > 0$, $3n \leq c$

Therefore, $n \leq c/3$

This cannot hold for all sufficiently large n, because n can increase without limit while c remains constant.

Thus, the assumption is false.

$f(n) \notin O(n)$

Conclusion: The function has quadratic growth, not linear growth. Its dominant term is $3n^2$.

(G) SCALABILITY DECISION FOR ONE MILLION PRODUCTS

Given data

• Number of products: n = 1,000,000

• Processing rate: 10^8 basic operations per second

• Nightly processing limit: 600 seconds (10 minutes)

The estimated running time is

Running time = Number of basic operations / 10^8

Step 1: M1 – Duplicate-Listing Detector

The exact number of comparisons is $C(n) = n(n-1)/2$

Substituting n = 1,000,000,

C(n) = 1,000,000(999,999)/2

C(n) = 499,999,500,000

Calculate the running time:

t = 499,999,500,000 / 10^8

t = 4,999.995 seconds

Convert into minutes: t = 4,999.995 / 60

t ≈ 83.33 minutes

Decision: Fails the 10-minute limit.

Recommendation: Replace the nested-loop comparison with hash-based grouping using normalized product titles and seller IDs. This can reduce expected complexity from $\Theta(n^2)$ to $\Theta(n)$, assuming efficient hashing.

Step 2: M2 – API Rate Limiter

The operation count is $A(n) = 2^{\lceil \log_2 n \rceil} - 1$

Since, $\lceil \log_2(1,000,000) \rceil = 20$

$A(n) = 2^{20} - 1 = 1,048,575$

Running time: t = 1,048,575 / 10^8

t ≈ 0.01049 seconds

Decision: Meets the 10-minute limit. No complexity-based redesign is required.

Step 3: M3 – Product Search S2 (Binary Search)

Binary search requires approximately 20 comparisons in the worst case.

$C(n) \approx \lceil \log_2(1{,}000{,}000) \rceil$

$C(n) \approx 20$

Running time: $t \approx 20 / 10^8$

$$\mathbf{t \approx 0.0000002 \text{ seconds}}$$

Decision: Easily meets the processing limit under the given assumptions.

Recommendation: Retain binary search and keep the product index sorted and updated. Actual response times will be higher because of memory access, network requests, and other system overheads.

Step 4: M4 – Current Recommendation Engine

Given: $T(n) = 4T(n/2) + n$

From the Master Theorem, $T(n) = \Theta(n^2)$

Using n^2 as an order-of-growth estimate,

Operations $\approx (1{,}000{,}000)^2$

Operations $\approx 10^{12}$

Running time: $t \approx 10^{12} / 10^8$

$t \approx 10{,}000$ seconds

Convert into minutes: $t = 10{,}000 / 60$

$$\mathbf{t \approx 166.67 \text{ minutes}}$$

Decision: Fails the 10-minute limit.

Recommendation: Redesign the recommendation engine to reduce unnecessary replication and use the proposed divide-and-conquer approach.

Step 5: M4 – Proposed Recommendation Engine

The proposed recurrence is $T(n) = 2T(n/2) + n$

Its complexity is $T(n) = \Theta(n \log n)$

For n = 1,000,000, Operations ≈ $n \log_2 n$

Operations ≈ 1,000,000(19.93157)

Operations ≈ 19,931,570

Running time: $t \approx 19{,}931{,}570 / 10^8$

$t \approx 0.199$ seconds

Decision: Meets the 10-minute limit under the given processing assumptions.

Step 6: M5 – Sales Analytics Engine

Given: $T(n) = 2T(n/2) + n \log n$

Its complexity is $T(n) = \Theta(n \log^2 n)$

Using the reference document's order-of-growth estimate,

Operations ≈ $n(\log_2 n)^2$

Operations ≈ $1{,}000{,}000(19.93157)^2$

Operations ≈ 397,266,000

Running time: $t \approx 397{,}266{,}000 / 10^8$

$t \approx 3.97$ seconds

Decision: Meets the 10-minute limit under the given processing assumptions.

Recommendation: Retain M5, avoid unnecessary repeated sorting, and monitor actual execution time as the catalogue grows. The operation count is an estimate, not an exact count of every comparison and data movement.

Step 7: Comparison of all modules

Module	**Estimated operations**	**Estimated time**	**Decision**
M1 – Duplicate detector	4.999995×10^{11}	83.33 min	Fails
M2 – API rate limiter	1,048,575	0.01049 sec	Meets
M3 – Binary search	About 20 comparisons per	0.0000002 sec per search	Meets

	search		
M4 – Current design	Order of 10^{12}	166.67 min	Fails
M4 – Proposed design	About 1.993×10^7	0.199 sec	Meets
M5 – Sales analytics	About 3.973×10^8	3.97 sec	Meets

Note: The timings are idealized estimates based on 10^8 operations per second. M1 and M4's current-design times use order-of-growth estimates, and the M3 figure represents one search rather than a full batch of searches.

FINAL SCALABILITY RECOMMENDATIONS

• Redesign M1: Replace the nested-loop duplicate detector with hash-based grouping to improve expected complexity from $\Theta(n^2)$ to $\Theta(n)$.

• Redesign M4: Replace the current recommendation algorithm with the proposed divide-and-conquer approach, improving complexity from $\Theta(n^2)$ to $\Theta(n \log n)$.

• Retain M2: Its tight bound is $\Theta(n)$, and its estimated workload is well within the processing limit.

• Retain M3 S2: Binary search provides $\Theta(\log n)$ search time when the product index is sorted and maintained.

• Maintain M5: Its $\Theta(n \log^2 n)$ complexity is expected to fit the nightly processing window under the stated

assumptions. Avoid redundant sorting and monitor performance.

OVERALL CONCLUSION

The scalability audit shows that algorithm selection strongly affects ShopSphere's performance. At one million products, the duplicate-listing detector (M1) and the current recommendation engine (M4) are expected to exceed the 10-minute nightly processing window.

Redesigning M1 using hash-based grouping and M4 using the proposed divide-and-conquer approach can significantly reduce processing time. M2, M3 S2, and M5 can be retained, with performance monitoring where appropriate.

These improvements will help ShopSphere accommodate catalogue growth while maintaining efficient duplicate detection, product search, recommendation generation, and sales analytics.